Electronics & Comm. Dept.
Faculty of Engineering
Cairo University
2025-2026

قسم الالكترونيات و الاتصالات
كلية الهندسة
جامعة القاهرة

OFDM Project

Fourth Year Communications Report

T 38

Name	Section	ID	BN
بسنت نبيل فوزي محمد	1	9220201	39
جمال علاء الدين عبدالمنعم محمد	2	9220224	46
سهيله احمد محمد عبدالمجيد	2	9220377	71

Submitted to:

Dr/ Mohamed Khairy

TA/ Mohamed Khaled

Table of Contents

Problem 1: 4

- Code Output 4
- Part C answer 4

Problem 2: 5

Part A: BPSK Scheme 8

- MATLAB Code Explanation 8
- Output Figure 8
- Comment 8

Part B: QPSK Scheme 9

- MATLAB code Explanation 9
- Output Figure 9
- Comment 9

Part C: 16 QAM Scheme 11

- MATLAB code Explanation 11
- Output Figure 11
- Comments 12

BER Performance Comparison of BPSK, QPSK, and 16-QAM over Rayleigh Channel 12

- Output Figure 12
- Comment 12

Problem 3: 13

Part A: QPSK Scheme 15

- MATLAB code Explanation 15
- Output Figure 15
- Comment 15

Part B: 16 QAM Scheme 16

- MATLAB code Explanation 16
- Output Figure 16
- Comment 16

Comparison between Flat Fading and Frequency Selective 17
Appendix: Full MATLAB Code 18

List of Figures

Figure 1: DFT vs FFT computation 4
Figure 2: BPSK over Rayleigh Channel 8
Figure 3: QPSK over Rayleigh Channel 9
Figure 4: 16-QAM over Rayleigh Channel 11
Figure 5 : BPSK, QPSK, and 16-QAM over Rayleigh Channel 12
Figure 6 : OFDM QPSK – Flat & Frequency-Selective Rayleigh Channel 15
Figure 7: OFDM 16QAM - Flat Fading vs Frequency Selective Rayleigh Channel 16

Problem 1: Execution time of DFT and FFT

In this section, it is required to compare between the execution time of Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT).

a. Write MATLAB function to implement DFT according to

$$X(k) = \sum_{n=0}^{N-1} x(n)\, e^{-\frac{j2\pi nk}{N}} \quad 0 \leq k \leq N-1$$

b. Generate random test signal, xi(n) of length L = 4096, then use tic and toc commands in MATLAB to compute the execution time of DFT {xi(n)} using the function developed in part a. and the execution time of FFT {xi(n)} using the built-in MATLAB function FFT( ).

c. Identify which of the investigated transform implementations offers superior performance with respect to the execution time.

➢ Code Output

```
DFT execution time: 0.1730 seconds
FFT execution time: 0.0003 seconds
FFT is faster than DFT.
fx >>
```

Figure 1: DFT vs FFT computation

➢ Part C answer

1. **DFT (Discrete Fourier Transform)**
    - Implemented using either matrix multiplication or nested loops.
    - Computational complexity: **O(N²)**.
    - For L = 4096, this involves over 16 million calculations.
2. **FFT (Fast Fourier Transform)**
    - Built-in MATLAB fft() function.
    - Computational complexity: **O(N log N)**.
    - For L = 4096, this is roughly 50,000 calculations.

So **FFT offers better performance** in terms of execution time.

Problem 2: Bit-error rate performance for BPSK and 16-QAM over Rayleigh flat fading channel

In this section it is required to simulate the Bit-Error Rate (BER) performance of BPSK when transmitting over Rayleigh flat fading channel. The system model for BPSK is shown in Fig. 1

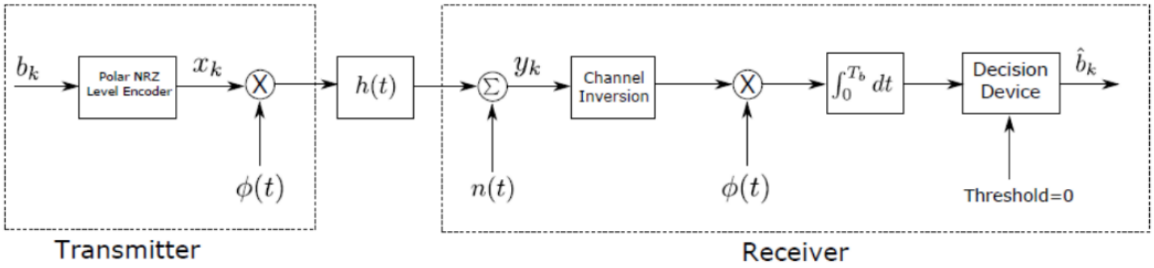


Figure 1: BPSK System Model

Where $b_k \in \{0,1\}$represents the binary data and
$x_k \in \{\sqrt{E_b}, -\sqrt{E_b}\}$ denotes the baseband BPSK symbols.

The carrier waveform $\phi(t)$is defined as

$$\phi(t) = \sqrt{\frac{2}{T_b}} \cos(2\pi f_c t)$$

The channel impulse response $h(t)$is given by

$$h(t) = A_h e^{j\theta_h} \delta(t)$$

Equivalently, the channel impulse response can be written as

$$h(t) = (h_r + jh_i)\delta(t)$$

- The amplitude of the channel impulse response $A_h$is a random variable drawn from a **Rayleigh distribution**.
- The phase $\theta_h$is a random variable uniformly distributed over the interval $[0, 2\pi]$.
- The real part of the channel impulse response $h_r$is a **Gaussian random variable** with zero mean and variance equal to $\frac{1}{2}$.
- The imaginary part of the channel impulse response $h_i$is a **Gaussian random variable** with zero mean and variance equal to $\frac{1}{2}$.

The Additive White Gaussian Noise (AWGN) $n(t)$is given by

$$n(t) = n_c(t)\cos(\omega_c t) - n_s(t)\sin(\omega_c t)$$

- $n_c(t)$is a random process such that, at any specified time instant t_1, $n_c(t_1)$is a Gaussian random variable with zero mean and variance equal to $\frac{N_0}{2}$.
- $n_s(t)$is a random process such that, at any specified time instant t_1, $n_s(t_1)$is a Gaussian random variable with zero mean and variance equal to $\frac{N_0}{2}$.

The overall system model can be expressed as

$$y_k = (h_r + jh_i)_k x_k + (n_c + jn_s)_k, k = 0,1,2, \dots$$

Steps of BPSK BER Simulation over a Rayleigh Fading Channel

a. Generate a random bit stream $b_k = [0,1,0,1, \dots]$

b. Generate BPSK symbols based on the bit stream $x_k = [-\sqrt{E_b}, \sqrt{E_b}, -\sqrt{E_b}, \sqrt{E_b}, \dots]$

c. Generate the complex channel vector $(h_r + jh_i)$and the complex noise vector $(n_c + jn_s)$according to the distributions explained above.

d. Compute the received symbol vector $y_k = (h_r + jh_i)_k x_k + (n_c + jn_s)_k, k = 0,1,2, \dots$

e. Compensate for the channel gain at the receiver (assuming perfect channel knowledge), apply a correlator, and perform hard-decision decoding to estimate the transmitted bit stream $\hat{b}_k$.

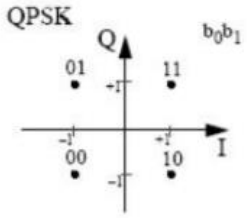
f. Compute the bit-error rate (BER) for a given signal-to-noise ratio (SNR), SNR $= \frac{E_b}{N_0}$.

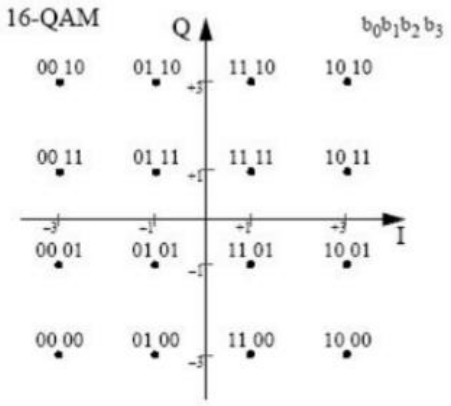
g. Change the noise level and repeat steps (a) through (f).

h. h. Plot the BER versus SNR.

i. Repeat the above steps using a **rate** $1/5$**repetition code**, where each transmitted "1" is represented by five consecutive "1"s and each transmitted "0" is represented by five consecutive "0"s.

Repeat the entire simulation procedure using **QPSK** and **16-QAM** modulation schemes with their corresponding constellations.





Part A: BPSK Scheme

➢ MATLAB Code Explanation

Random bits are generated and then BPSK modulated (0 -> -A, 1 -> A) then transmitted through a complex Rayleigh fading channel and corrupted by additive white Gaussian noise. At the receiver, perfect channel knowledge is assumed, enabling channel equalization followed by hard-decision detection. A rate 1/5 repetition code is applied by transmitting each information bit five times and recovering it using hard-decision majority logic. BER performance is evaluated for both constant energy per information bit and constant energy per transmitted bit versus E_b/N_0 to illustrate the gain achieved through repetition coding.

The complete MATLAB code for the BPSK simulation is provided at the end of the report.

➢ Output Figure

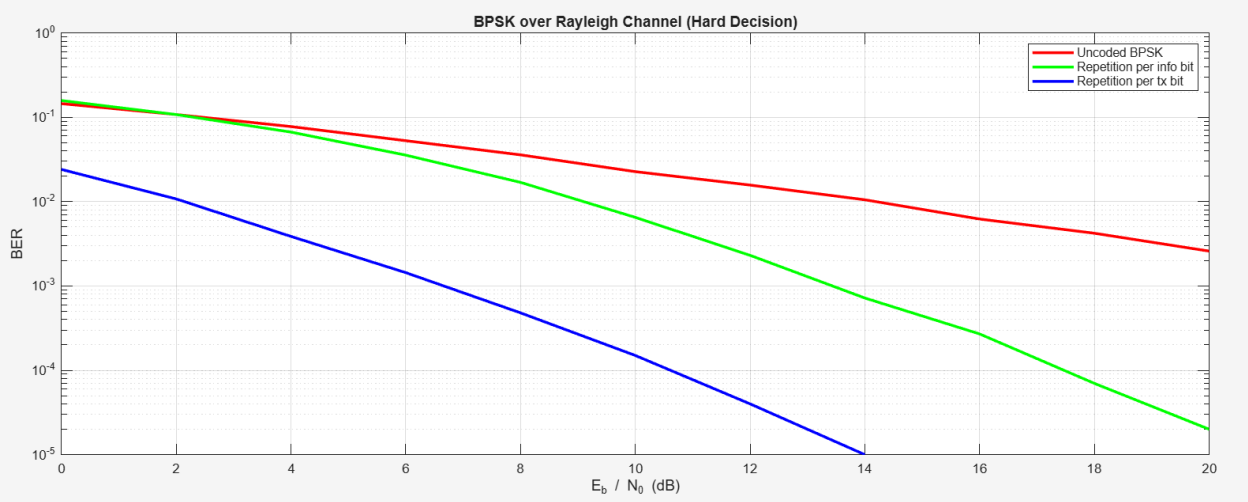


Figure 2: BPSK over Rayleigh Channel

➢ Comment

The BER decreases with increasing $\frac{E_b}{N_0}$ for all cases, confirming correct BPSK operation over a Rayleigh fading channel. Uncoded BPSK shows the highest BER due to the absence of redundancy. Applying a rate 1/5 repetition code improves performance through majority decoding, with the lowest BER achieved when measured per transmitted bit, as the higher transmitted energy provides a clear coding gain over both uncoded BPSK and repetition per information bit.

Part B: QPSK Scheme

➢ MATLAB code Explanation

The main difference between the BPSK and QPSK implementations lies in the mapping and demapping stages, while the rest of the simulation including Rayleigh channel generation, AWGN addition, channel equalization, and repetition coding remain the same.

In QPSK, two information bits are mapped to one complex symbol, which provides higher spectral efficiency compared to BPSK. Therefore, dedicated mapper and demapper functions are implemented to perform the bit-to-symbol conversion at the transmitter and the symbol-to-bit recovery at the receiver.

The complete MATLAB code for the QPSK simulation is provided at the end of the report.

Mapper Code Explanation: The QPSK mapper groups the input bit stream into pairs of bits, where each pair is mapped to one complex symbol using gray mapping. The constellation points are located in the four quadrants of the complex plane, with each symbol having equal energy.

Demapper Code Explanation: At the receiver, after channel equalization, the QPSK demapper performs hard-decision detection. The real and imaginary parts of each received symbol are examined to determine the quadrant in which the symbol lies. Based on this quadrant, the corresponding two transmitted bits are recovered.

➢ Output Figure

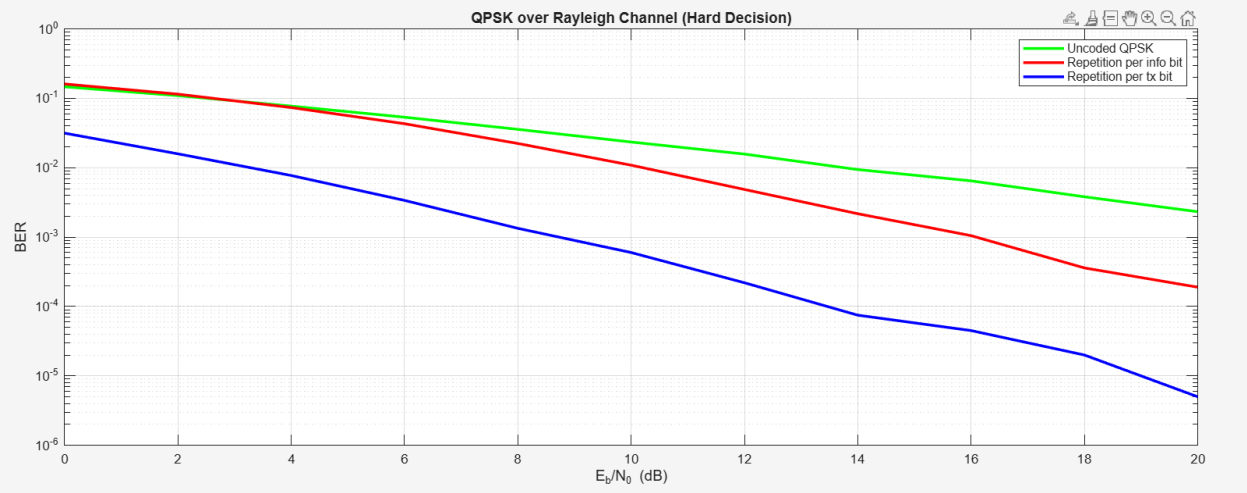


Figure 3: QPSK over Rayleigh Channel

➢ Comment

The BER decreases as E_b/N_0 increases for all cases, which confirms correct QPSK operation over a Rayleigh fading channel. Uncoded QPSK shows the highest BER since each bit is transmitted only once, making the system highly sensitive to deep fades and noise. When a

rate 1/5 repetition code is used, performance improves because multiple copies of each bit help reduce the effect of deep fades through majority decoding. The best performance is observed for repetition per transmitted bit, as the symbol energy is not reduced, resulting in higher received energy and a clear coding gain compared to the other cases.

Part C: 16 QAM Scheme

➢ MATLAB code Explanation

Compared to BPSK, the 16-QAM implementation differs primarily in the mapping and demapping stages. BPSK maps one bit to a real-valued symbol using simple threshold detection, whereas 16-QAM maps four bits to a complex symbol using a predefined constellation and minimum-distance demapping. All other simulation blocks, including Rayleigh fading, AWGN addition, channel equalization, repetition coding, and BER evaluation, follow the same structure. Therefore, only the modulation-specific mapping and demapping operations are explaned, while the complete MATLAB code is provided at the end of the report.

- In the transmitter, the input bit stream is grouped into sets of four bits. Each group is converted into a symbol index and mapped to a corresponding complex point in the predefined 16-QAM constellation for transmission.
- At the receiver, hard-decision demapping is performed on the equalized symbols. For each received symbol, the Euclidean distance to all constellation points is computed, and the closest point is selected. The corresponding symbol index is then converted back into the original four-bit group to recover the transmitted bit stream, which is subsequently used to calculate the BER.

➢ Output Figure

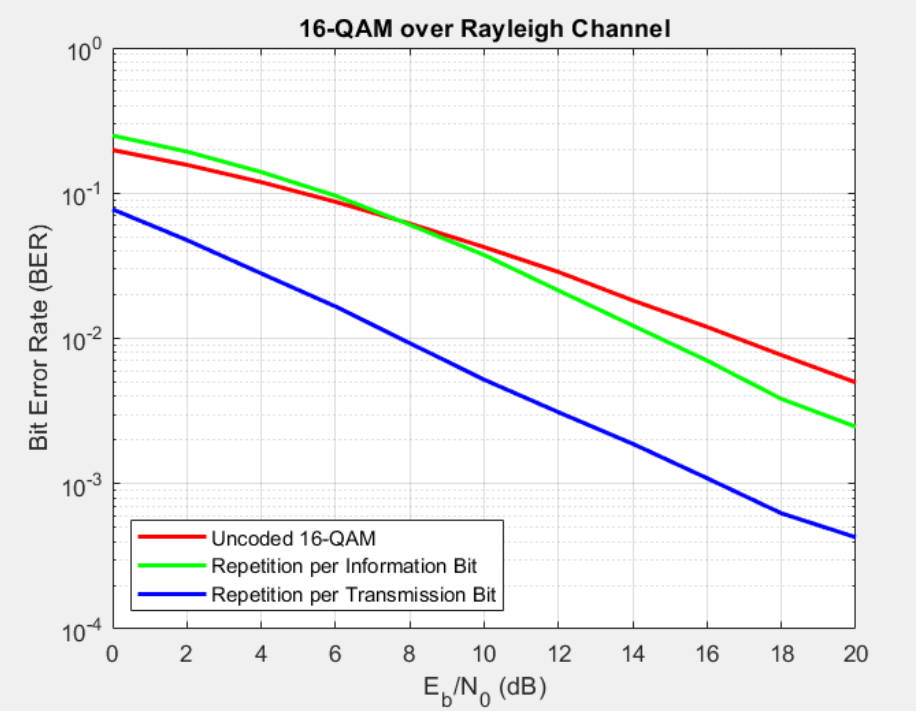


Figure 4: 16-QAM over Rayleigh Channel

➢ Comments

- Repetition with the same energy per information bit provides limited improvement, since the energy is spread over multiple repeated bits.
- Repetition with the same energy per transmitted bit achieves the best performance due to the effective increase in energy per information bit and diversity gain.

BER Performance Comparison of BPSK, QPSK, and 16-QAM over Rayleigh Channel

➢ Output Figure

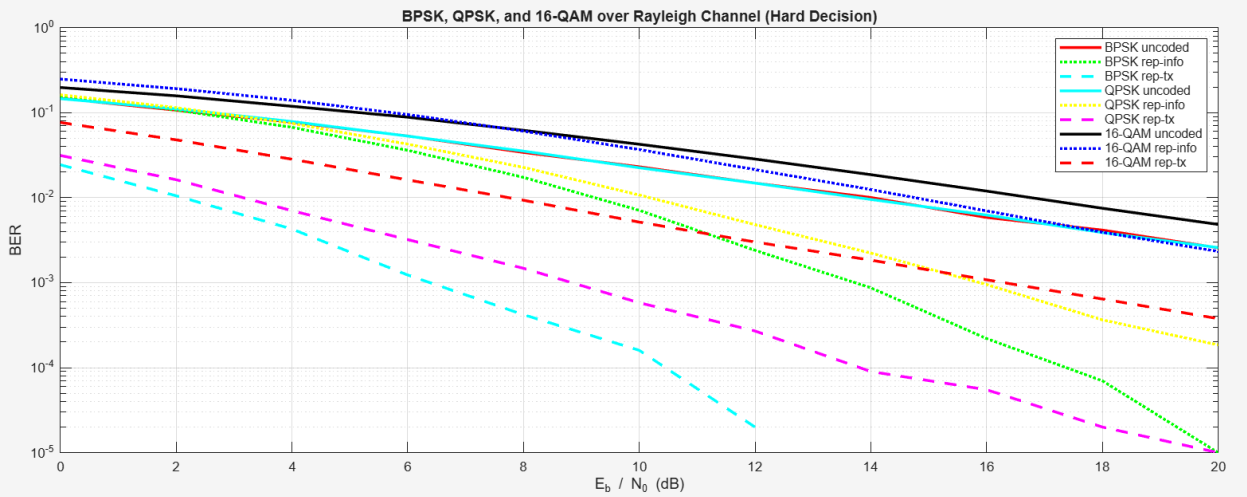
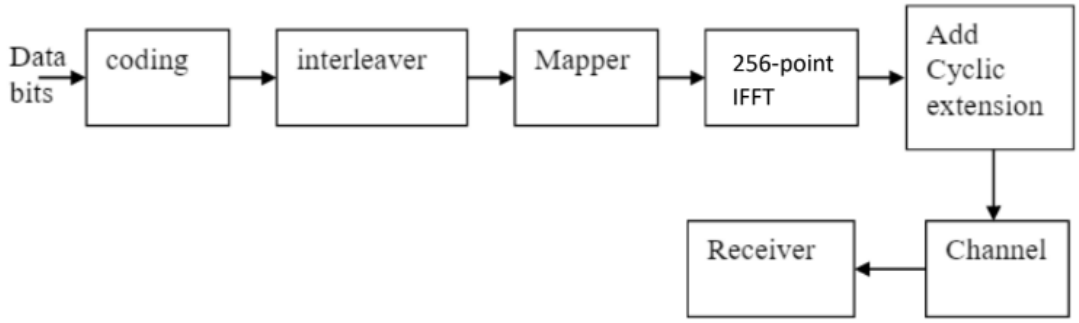


Figure 5 : BPSK, QPSK, and 16-QAM over Rayleigh Channel

➢ Comment

- For all cases, the BER decreases as E_b/N_0 increases. The uncoded BPSK and QPSK curves overlap, showing identical BER performance when compared at the same E_b/N_0, while 16-QAM has a higher BER due to its denser constellation. At low E_b/N_0, uncoded transmission can outperform repetition coding per information bit, since the reduction in symbol energy limits the coding gain under strong noise conditions.
- At higher E_b/N_0, repetition coding becomes more effective, with repetition per transmitted bit achieving the lowest BER. In this case, BPSK provides the best performance, followed by QPSK and then 16-QAM, due to the increased total transmitted energy and the higher noise sensitivity of higher-order modulation.

Problem 3: OFDM system simulation



The above blocks are described as follows:

1. Coding:

Either no coding or rate 1/5 repetition code are used. Note that you have to adjust the number of input bits per OFDM symbol when using repetition code. For example, if you use QPSK, only 51 data bits will be used per symbol, 1 zero is added to the encoded data to have 256 bits at the input of the mapper before the IFFT block.

2. Interleaver:

- For QPSK, the size of the interleaver is 32 by 16.
- For 16QAM, the interleaver size is 32 by 32.

3. Mapper:

The mappers used are the same as those in the single carrier system above.

4. IFFT:

Use a size 256 IFFT block. In MATLAB use the command "IFFT"

5. Channel:

Two channel models should be considered:

a. Rayleigh flat fading channel: Same as single carrier system

b. Frequency selective Fading channel: In order to model some type of frequency selective fading channel, assume that every sub-channel is independently faded by a different Rayleigh fading channel. Note that this fading should be modeled in the frequency domain.

6. Receiver:

Design a receiver to receive the signal described above in the two cases of the channel model.

Requirements:

Plot the BER against SNR for BPSK, and 16-QAM modulation schemes, for the two channel models mentioned above and for coding and no coding scenarios (4 cases for each modulation scheme).

Part A: QPSK Scheme

➢ MATLAB code Explanation

The code simulates an OFDM system with QPSK modulation over flat and frequency-selective Rayleigh fading channels, considering both uncoded transmission and rate 1/5 repetition coding.

For each OFDM symbol, random bits are generated and mapped to QPSK symbols, followed by IFFT processing and cyclic prefix insertion. In the flat fading case, a single Rayleigh fading coefficient affects the entire OFDM symbol, whereas in the frequency-selective case, independent Rayleigh fading is applied to each subcarrier in the frequency domain. AWGN is added in both cases.

At the receiver, the cyclic prefix is removed, FFT-based equalization is performed, and hard-decision demapping is used to recover the transmitted bits. For the coded case, deinterleaving and majority-vote decoding are applied before BER computation.

The complete MATLAB code for this simulation is provided at the end of the report.

➢ Output Figure

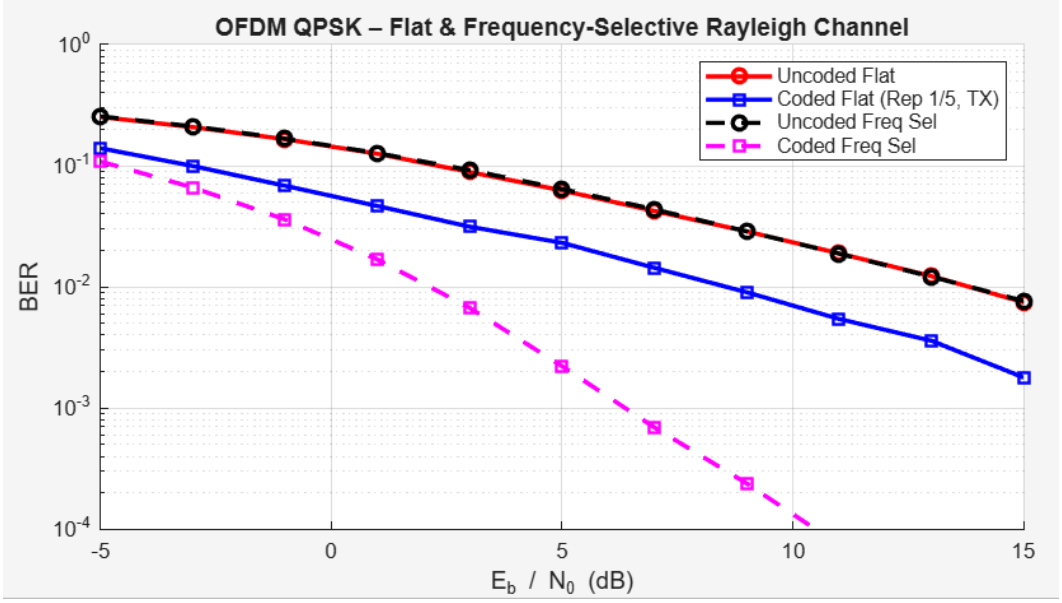


Figure 6 : OFDM QPSK – Flat & Frequency-Selective Rayleigh Channel

➢ Comment

Repetition coding (same energy per tx bits) improves BER in both flat and frequency-selective channels, but the gain is much larger in frequency-selective channels. In frequency-selective fading, some subcarriers experience deep fades, and coding spreads each bit across multiple symbols, allowing recovery even from faded subcarriers.

Part B: 16 QAM Scheme

➢ MATLAB code Explanation

This part of the code simulates an OFDM system using 16-QAM modulation over flat and frequency-selective Rayleigh fading channels, considering both uncoded transmission and rate 1/5 repetition coding.

- At the transmitter, random bits are grouped into sets of four and mapped to complex 16-QAM symbols using a predefined constellation. The symbols are converted to the time domain using an IFFT, and a cyclic prefix is added. In the flat fading case, a single Rayleigh coefficient affects the entire OFDM symbol, while in the frequency-selective case, independent Rayleigh fading is applied to each subcarrier in the frequency domain.
- At the receiver, the cyclic prefix is removed, FFT-based equalization is performed, and hard-decision demapping based on minimum distance is used to recover the transmitted bits. For the coded case, deinterleaving and majority-vote decoding are applied before BER computation.

➢ Output Figure

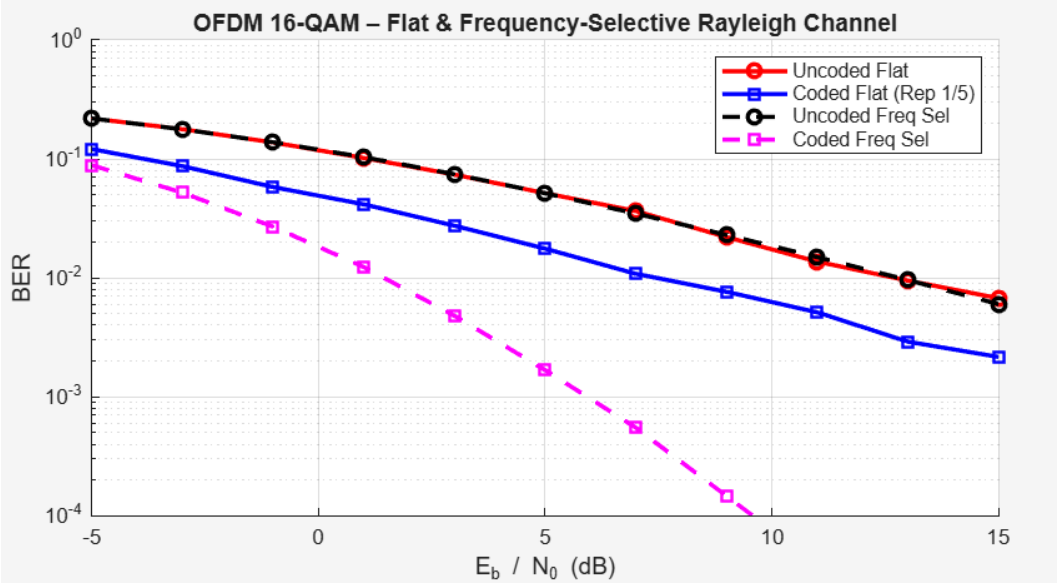


Figure 7: OFDM 16QAM - Flat Fading vs Frequency Selective Rayleigh Channel

➢ Comment

Repetition coding greatly reduces BER, with the largest gain in frequency-selective channels, where it helps recover bits affected by deep fades on some subcarriers.

Comparison between Flat Fading and Frequency Selective

	Flat Fading	Frequency-Selective Fading
Impact on OFDM	Each OFDM subcarrier sees the **same fading**; no subcarrier experiences deep fading independently.	Different OFDM subcarriers can fade independently; some subcarriers may be deeply faded while others remain strong.
BER Performance	Better for uncoded signals; simpler equalization (just divide by h).	More challenging for uncoded signals; equalization per subcarrier is needed. Coding and interleaving significantly improve performance.
OFDM Advantage	OFDM is **not strictly necessary**.	OFDM **converts frequency-selective fading into multiple flat-fading subcarriers**, greatly reducing ISI.
Channel Modeling	Single Rayleigh coefficient.	Multi-tap Rayleigh model; often modeled with **independent fading per subcarrier**.
Simulation Example	`rx = h*tx + n` same h for all subcarriers.	`Yf = fft(rx_no_cp)/h_freq` independent h_freq per subcarrier.

Conclusion: With coding and interleaving, frequency-selective fading often outperforms flat fading because of frequency diversity. Different subcarriers fade independently, so some bits avoid deep fades, unlike flat fading where all bits are affected by the same channel attenuation.

Appendix: Full MATLAB Code

```
clc; clear; close all;

while true
    fprintf('\n================ SELECT PART ================\n');
    fprintf('Press 1  → Run Part 1 (Execution time of DFT and FFT)\n');
    fprintf('Press 2  → Run Part 2a (BPSK Scheme over Rayleigh)\n');
    fprintf('Press 3  → Run Part 2b (QPSK Scheme over Rayleigh)\n');
    fprintf('Press 4  → Run Part 2c (16QAM Scheme over Rayleigh)\n');
    fprintf('Press 5  → Run Part 3a (QPSK Scheme OFDM)\n');
    fprintf('Press 6  → Run Part 3b (16QAM Scheme OFDM)\n');
    fprintf('Press 0  → EXIT\n');
    fprintf('============================================\n');

    choice = input('Enter part number: ');

    switch choice
        case 1
            part1();
        case 2
            part2();
        case 3
            part3();
        case 4
            part4();
        case 5
            part5();
        case 6
            part6();
        case 0
            fprintf('Exiting program...\n');
            break;   % exit the while loop
        otherwise
            fprintf('? Invalid choice. Enter a number from 0 to 6.\n');
    end

    fprintf('\nPress any key to return to the main menu...\n');
    pause;
    clc;
end

%% part 1: (Execution time of DFT and FFT)
function part1()
    %% Clear workspace and command window
    clear; clc;
    %% Part (b): Generate random test signal
    L = 4096;                   % signal length
    x = rand(1, L);             % random signal

    %% Compute execution time for DFT
    tic;
    X_dft = myDFT(x);
    time_dft = toc;
```

```
    fprintf('DFT execution time: %.4f seconds\n', time_dft);

    %% Compute execution time for FFT (for comparison)
    tic;
    X_fft = fft(x);
    time_fft = toc;
    fprintf('FFT execution time: %.4f seconds\n', time_fft);

    %% Identify which is faster
    if time_dft > time_fft
        fprintf('FFT is faster than DFT.\n');
    else
        fprintf('DFT is faster than FFT (unexpected).\n');
    end
    %% ----------------- DFT Function-----------------
    function X = myDFT(x)
        % DFT implementation
        N = length(x);                  % signal length
        n = 0:N-1;                      % row vector of n
        k = n';                          % column vector of k
        W = exp(-1j*2*pi*k*n/N);        % DFT matrix
        X = W * x(:);                    % matrix multiplication
    end

end



%% part 2: (BPSK Scheme over Rayleigh)
function part2()
    %% ====================== BPSK======================
    clear; clc;

    %% ---------------- Parameters ----------------
    Number_of_Information_Bits_BPSK = 100000;
    A = 1;
    Eb_BPSK = A^2;
    Rep = 5;

    Eb_No = 0:2:20;
    Eb_No_linear = 10.^(Eb_No/10);

    %% ---------------- Bit Generation ----------------
    Information_bits_BPSK = randi([0
1],1,Number_of_Information_Bits_BPSK);

    %% ---------------- BPSK Mapping ----------------
    Mapped_BPSK = (2*Information_bits_BPSK - 1)*A;
    %% ---------------- BER  ----------------
    BER_uncoded  = zeros(1,length(Eb_No));
    BER_rep_info = zeros(1,length(Eb_No));
    BER_rep_tx   = zeros(1,length(Eb_No));
    %% ====================== SNR Loop ======================
```

```
        for idx = 1:length(Eb_No)

        No_BPSK = Eb_BPSK / Eb_No_linear(idx);

        %% ================ Uncoded BPSK ================
        % Rayleigh channel
        hr = sqrt(0.5) * randn(1, Number_of_Information_Bits_BPSK);
        hi = sqrt(0.5) * randn(1, Number_of_Information_Bits_BPSK);
        h = hr + 1j*hi;

        % AWGN noise
        nc = sqrt(No_BPSK/2) * randn(1,
Number_of_Information_Bits_BPSK);
        ns = sqrt(No_BPSK/2) * randn(1,
Number_of_Information_Bits_BPSK);
        n = nc + 1j*ns;

          % Received signal
        Rx = h .* Mapped_BPSK + n;

        % Channel equalization
        Rx_eq = Rx ./ h;

        % Hard decision
        DeMapped_Signal = real(Rx_eq) > 0;
        BER_uncoded(idx) = sum(DeMapped_Signal ~= Information_bits_BPSK)
/ ...
                           Number_of_Information_Bits_BPSK;

        %% ===== Repetition per INFORMATION bit =====
        Repeated_bits = repelem(Information_bits_BPSK,Rep);
        Number_of_Repeated_Bits = length(Repeated_bits);
        % Normalize amplitude to keep Eb per information bit constant
        A_rep_info = A ./sqrt(Rep);
        Mapped_rep_info = A_rep_info*(2*Repeated_bits - 1);

        h = sqrt(0.5)*(randn(1,Number_of_Repeated_Bits) + ...
                       1j*randn(1,Number_of_Repeated_Bits));

        noise = sqrt(No_BPSK/2)*(randn(1,Number_of_Repeated_Bits) + ...
                                 1j*randn(1,Number_of_Repeated_Bits));

        Rx = h .* Mapped_rep_info + noise;
        Rx_eq = Rx ./ h;

        % Hard-decision demodulation: decide bit = 1 if the received
signal > 0,
        %otherwise decide bit = 0
        DeMapped_Signal = real(Rx_eq) > 0;

        Restored_Signal = zeros(1,Number_of_Information_Bits_BPSK);
        for k = 1:Number_of_Information_Bits_BPSK
            group = DeMapped_Signal((k-1)*Rep+1:k*Rep);
```

```
            % Majority-vote decoding for repetition code: decide bit = 1
if at least
            % half of the repeated bits are 1, otherwise decide bit = 0
            Restored_Signal(k) = sum(group) >= ceil(Rep/2);
        end
        BER_rep_info(idx) = sum(Restored_Signal ~=
Information_bits_BPSK) / ...
                            Number_of_Information_Bits_BPSK;

        %% ===== Repetition per TRANSMISSION bit  =====
        Mapped_rep_tx = A*(2*Repeated_bits - 1);

        h = sqrt(0.5)*(randn(1,Number_of_Repeated_Bits) + ...
                       1j*randn(1,Number_of_Repeated_Bits));

        noise = sqrt(No_BPSK/2)*(randn(1,Number_of_Repeated_Bits) + ...
                                 1j*randn(1,Number_of_Repeated_Bits));

        Received = h .* Mapped_rep_tx + noise;
        Received = Received ./ h;

        DeMapped_Signal = real(Received) > 0;

        Restored_Signal = zeros(1,Number_of_Information_Bits_BPSK);
        for k = 1:Number_of_Information_Bits_BPSK
            group = DeMapped_Signal((k-1)*Rep+1:k*Rep);
            Restored_Signal(k) = sum(group) >= ceil(Rep/2);
        end
        BER_rep_tx(idx) = sum(Restored_Signal ~= Information_bits_BPSK)
/ ...
                          Number_of_Information_Bits_BPSK;

    end

    %% ====================== Plot ======================
    figure;
    semilogy(Eb_No,BER_uncoded,'r','LineWidth',2); hold on;
    semilogy(Eb_No,BER_rep_info,'g','LineWidth',2);
    semilogy(Eb_No,BER_rep_tx,'b','LineWidth',2);
    grid on;
    xlabel('E_b / N_0 (dB)');
    ylabel('BER');
    legend('Uncoded BPSK','Repetition per info bit','Repetition per tx
bit');
    title('BPSK over Rayleigh Channel (Hard Decision)');

end



%% part 3: (QPSK Scheme over Rayleigh)
function part3()
    %% --------------- Parameters ---------------
```

```
    Number_of_Information_Bits_QPSK = 100000;
    A = 1;
    Eb_QPSK = A^2;
    Rep = 5;

    Eb_No = 0:2:20;
    Eb_No_linear = 10.^(Eb_No/10);

    %% ====================== QPSK ======================

    num_bits_QPSK = 200000;
    data_bits_QPSK = randi([0 1],1,num_bits_QPSK);

    tx_symbols_QPSK = qpsk_mapper(data_bits_QPSK,A);
    num_symbols_QPSK = length(tx_symbols_QPSK);

    BER_QPSK_uncoded  = zeros(1,length(Eb_No));
    BER_QPSK_rep_info = zeros(1,length(Eb_No));
    BER_QPSK_rep_tx   = zeros(1,length(Eb_No));

    for idx = 1:length(Eb_No)

        N0 = Eb_QPSK / Eb_No_linear(idx);

        %% -------- Uncoded QPSK --------
        h = sqrt(0.5)*(randn(1,num_symbols_QPSK) + ...
                       1j*randn(1,num_symbols_QPSK));
        n = sqrt(N0/2)*(randn(1,num_symbols_QPSK) + ...
                        1j*randn(1,num_symbols_QPSK));

        Rx = h.*tx_symbols_QPSK + n;
        Rx_eq = Rx./h;

        demapped_bits = qpsk_demapper(Rx_eq);
        BER_QPSK_uncoded(idx) = sum(demapped_bits ~=
data_bits_QPSK)/num_bits_QPSK;

        %% -------- Repetition per INFORMATION bit --------
        rep_bits = repelem(data_bits_QPSK,Rep);
        tx_rep = qpsk_mapper(rep_bits, A/sqrt(Rep));

        h = sqrt(0.5)*(randn(1,length(tx_rep)) + ...
                       1j*randn(1,length(tx_rep)));
        n = sqrt(N0/2)*(randn(1,length(tx_rep)) + ...
                        1j*randn(1,length(tx_rep)));

        Rx = h.*tx_rep + n;
        Rx_eq = Rx./h;

        demap_bits = qpsk_demapper(Rx_eq);
        restored = zeros(1,num_bits_QPSK);
        for k = 1:num_bits_QPSK
            group = demap_bits((k-1)*Rep+1:k*Rep);
            restored(k) = sum(group) >= ceil(Rep/2);
```

```
        end

        BER_QPSK_rep_info(idx) = sum(restored ~=
data_bits_QPSK)/num_bits_QPSK;

        %% -------- Repetition per TRANSMISSION bit --------
        tx_rep = qpsk_mapper(rep_bits, A);

        h = sqrt(0.5)*(randn(1,length(tx_rep)) + ...
                       1j*randn(1,length(tx_rep)));
        n = sqrt(N0/2)*(randn(1,length(tx_rep)) + ...
                        1j*randn(1,length(tx_rep)));

        Rx = h.*tx_rep + n;
        Rx_eq = Rx./h;

        demap_bits = qpsk_demapper(Rx_eq);
        restored = zeros(1,num_bits_QPSK);
        for k = 1:num_bits_QPSK
            group = demap_bits((k-1)*Rep+1:k*Rep);
            restored(k) = sum(group) >= ceil(Rep/2);
        end

        BER_QPSK_rep_tx(idx) = sum(restored ~=
data_bits_QPSK)/num_bits_QPSK;
    end

    %% ====================== FINAL PLOT (3 CURVES)
======================
    figure;

    semilogy(Eb_No,BER_QPSK_uncoded,'r','LineWidth',2);hold on;
    semilogy(Eb_No,BER_QPSK_rep_info,'g','LineWidth',2);
    semilogy(Eb_No,BER_QPSK_rep_tx,'b','LineWidth',2);

    grid on;
    xlabel('E_b / N_0 (dB)');
    ylabel('BER');
    legend( ...
        'QPSK Uncoded','QPSK Rep (info)','QPSK Rep (tx)' ...
    );
    title('QPSK over Rayleigh Channel (Hard Decision)');
    function symbols = qpsk_mapper(bits, A)

        bit_matrix = reshape(bits,2,[]).';
        num_symbols = size(bit_matrix,1);
        symbols = zeros(1,num_symbols);

        for i = 1:num_symbols
            b1 = bit_matrix(i,1);
            b2 = bit_matrix(i,2);

            if b1==0 && b2==0
                symbols(i) = -A - 1j*A;
```

```
            elseif b1==0 && b2==1
                symbols(i) = -A + 1j*A;
            elseif b1==1 && b2==1
                symbols(i) =  A + 1j*A;
            elseif b1==1 && b2==0
                symbols(i) =  A - 1j*A;
            end
        end
    end
    function demapped_bits = qpsk_demapper(symbols)
    % Hard decision demapper

        num_symbols = length(symbols);
        demapped_bits = zeros(1,2*num_symbols);

        for k = 1:num_symbols
            rp = real(symbols(k));
            ip = imag(symbols(k));

            if rp < 0 && ip < 0
                demapped_bits(2*k-1:2*k) = [0 0];
            elseif rp < 0 && ip >= 0
                demapped_bits(2*k-1:2*k) = [0 1];
            elseif rp >= 0 && ip >= 0
                demapped_bits(2*k-1:2*k) = [1 1];
            elseif rp >= 0 && ip < 0
                demapped_bits(2*k-1:2*k) = [1 0];
            end
        end
    end

end




%% part 4: (16QAM Scheme over Rayleigh)
function part4()
    %% ====================== 16-QAM ======================
    clear; clc;

    %% ---------------- Parameters ----------------
    num_bits_QAM = 400000;
    bits_per_symbol_QAM = 4;
    num_symbols_QAM = num_bits_QAM / bits_per_symbol_QAM;
    Rep = 5;

    Eb_QAM = 2.5;   %10(average symbol energy)/4
    Eb_No = 0:2:20;
    Eb_No_linear = 10.^(Eb_No/10);

    %% ---------------- Bit Generation ----------------
    data_bits_QAM = randi([0 1], 1, num_bits_QAM);
```

```
    bit_matrix_QAM = reshape(data_bits_QAM, bits_per_symbol_QAM, []).';
    symbol_indices_QAM = bi2de(bit_matrix_QAM, 'left-msb');

    %% ---------------- 16-QAM Constellation ----------------
    t_QAM = [ -3-3i, -3-1i, -3+3i, -3+1i, ...
              -1-3i, -1-1i, -1+3i, -1+1i, ...
               3-3i,  3-1i,  3+3i,  3+1i, ...
               1-3i,  1-1i,  1+3i,  1+1i ];

    Mapped_symbols_QAM = t_QAM(symbol_indices_QAM + 1).';

    %% ---------------- BER Storage ----------------
    BER_QAM_uncoded  = zeros(1,length(Eb_No));
    BER_QAM_rep_info = zeros(1,length(Eb_No));
    BER_QAM_rep_tx   = zeros(1,length(Eb_No));

    %% ====================== SNR LOOP ======================
    for idx = 1:length(Eb_No)

        N0 = Eb_QAM / Eb_No_linear(idx);

        %% ================= Uncoded 16-QAM =================
        h = sqrt(0.5)*(randn(num_symbols_QAM,1) + ...
                       1j*randn(num_symbols_QAM,1));

        noise = sqrt(N0/2)*(randn(num_symbols_QAM,1) + ...
                            1j*randn(num_symbols_QAM,1));

        Rx = h .* Mapped_symbols_QAM + noise;
        Rx_eq = Rx ./ h;

        Demapped_bits = zeros(1,num_bits_QAM);

        for j = 1:num_symbols_QAM
            distances = abs(Rx_eq(j) - t_QAM.');
            [~, idx_QAM] = min(distances);
            Demapped_bits(4*(j-1)+1:4*j) = de2bi(idx_QAM-1,4,'left-
msb');
        end

        BER_QAM_uncoded(idx) = sum(Demapped_bits ~= data_bits_QAM) / ...
                               num_bits_QAM;

        %% ===== Repetition per INFORMATION bit =====
        rep_bits = repelem(data_bits_QAM,Rep);
        num_bits_rep = length(rep_bits);
        num_symbols_rep = num_bits_rep / bits_per_symbol_QAM;

        bit_matrix_rep = reshape(rep_bits,bits_per_symbol_QAM,[]).';
        symbol_indices_rep = bi2de(bit_matrix_rep,'left-msb');

        % Scale constellation to keep Eb fixed
        t_rep_info = t_QAM / sqrt(Rep);
```

```
        tx_rep = t_rep_info(symbol_indices_rep + 1).';

        h = sqrt(0.5)*(randn(num_symbols_rep,1) + ...
                       1j*randn(num_symbols_rep,1));
        noise = sqrt(N0/2)*(randn(num_symbols_rep,1) + ...
                            1j*randn(num_symbols_rep,1));

        Rx = h .* tx_rep + noise;
        Rx_eq = Rx ./ h;

        DeMapped_bits = zeros(1,num_bits_rep);
        for j = 1:num_symbols_rep
            distances = abs(Rx_eq(j) - t_rep_info.');
            [~, idx_QAM] = min(distances);
            DeMapped_bits(4*(j-1)+1:4*j) = de2bi(idx_QAM-1,4,'left-
msb');
        end

        Restored_bits = zeros(1,num_bits_QAM);
        for b = 1:num_bits_QAM
            group = DeMapped_bits((b-1)*Rep+1:b*Rep);
            Restored_bits(b) = sum(group) >= ceil(Rep/2);
        end

        BER_QAM_rep_info(idx) = sum(Restored_bits ~= data_bits_QAM) /
...
                                num_bits_QAM;

        %% ===== Repetition per TRANSMISSION bit =====
        tx_rep = t_QAM(symbol_indices_rep + 1).';

        h = sqrt(0.5)*(randn(num_symbols_rep,1) + ...
                       1j*randn(num_symbols_rep,1));
        noise = sqrt(N0/2)*(randn(num_symbols_rep,1) + ...
                            1j*randn(num_symbols_rep,1));

        Rx = h .* tx_rep + noise;
        Rx_eq = Rx ./ h;

        DeMapped_bits = zeros(1,num_bits_rep);
        for j = 1:num_symbols_rep
            distances = abs(Rx_eq(j) - t_QAM.');
            [~, idx_QAM] = min(distances);
            DeMapped_bits(4*(j-1)+1:4*j) = de2bi(idx_QAM-1,4,'left-
msb');
        end

        Restored_bits = zeros(1,num_bits_QAM);
        for b = 1:num_bits_QAM
            group = DeMapped_bits((b-1)*Rep+1:b*Rep);
            Restored_bits(b) = sum(group) >= ceil(Rep/2);
        end

        BER_QAM_rep_tx(idx) = sum(Restored_bits ~= data_bits_QAM) / ...
```

```
                                num_bits_QAM;
    end

    %% ====================== BER PLOT ======================
    figure;
    semilogy(Eb_No, BER_QAM_uncoded, 'r', 'LineWidth', 1.8); hold on;
    semilogy(Eb_No, BER_QAM_rep_info, 'g', 'LineWidth', 1.8);
    semilogy(Eb_No, BER_QAM_rep_tx, 'b', 'LineWidth', 1.8);
    grid on;

    xlabel('E_b/N_0 (dB)');
    ylabel('Bit Error Rate (BER)');
    title('16-QAM over Rayleigh Channel');

    legend('Uncoded 16-QAM', ...
           'Repetition per Information Bit', ...
           'Repetition per Transmission Bit', ...
           'Location','southwest');



end




%% part 5: (QPSK Scheme OFDM)
function part5()
    clc; clear; close all;

    %% ================ PARAMETERS ================
    Nfft = 256;           % IFFT size
    Lcp  = 16;            % Cyclic prefix length
    Rep  = 5;             % Repetition factor
    EbN0_dB = -5:2:15;     % SNR range in dB
    EbN0 = 10.^(EbN0_dB/10);

    Nsym = 5000;           % OFDM symbols per SNR point

    %% ================ BER STORAGE ================
    BER_uncoded_flat = zeros(size(EbN0_dB));
    BER_coded_flat   = zeros(size(EbN0_dB));
    BER_uncoded_freq = zeros(size(EbN0_dB));
    BER_coded_freq   = zeros(size(EbN0_dB));

    %% ================ SNR LOOP ================
    for s = 1:length(EbN0_dB)

        err_unc_flat = 0; bits_unc_flat = 0;
        err_cod_flat = 0; bits_cod_flat = 0;
        err_unc_freq = 0; bits_unc_freq = 0;
        err_cod_freq = 0; bits_cod_freq = 0;

        N0 = 1 / EbN0(s);  % Eb = 1
```

```
        for sym = 1:Nsym
            %% ================================================
            %                   UNCODED QPSK - FLAT
            %% ================================================
            bits_uncoded = randi([0 1],1,512); % 2×256 bits
            Xf = qpsk_mapper(bits_uncoded,1);

            xt = ifft(Xf,Nfft);
            tx = [xt(end-Lcp+1:end) xt];

            h = (randn + 1j*randn)/sqrt(2);
            n = sqrt(N0/(2*Nfft))*(randn(size(tx)) +
1j*randn(size(tx)));

            rx = h*tx + n;
            rx = rx(Lcp+1:end);
            Yf = fft(rx,Nfft)/h;

            demapped_bits = qpsk_demapper(Yf);

            err_unc_flat  = err_unc_flat + sum(demapped_bits ~=
bits_uncoded);
            bits_unc_flat = bits_unc_flat + length(bits_uncoded);

            %% ================================================
            %          CODED QPSK (RATE 1/5 - TX BIT) - FLAT
            %% ================================================
            info_bits  = randi([0 1],1,floor(512/Rep)); % 102 bits
            coded_bits = repelem(info_bits,Rep);        % 510 bits
            coded_bits = [coded_bits 0 0];             % pad to 512

            % Interleaver (32×16)
            mat = reshape(coded_bits,32,16).';
            inter_bits = mat(:).';

            Xf = qpsk_mapper(inter_bits,1);  % TX-bit repetition

            xt = ifft(Xf,Nfft);
            tx = [xt(end-Lcp+1:end) xt];

            h = (randn + 1j*randn)/sqrt(2);
            n = sqrt(N0/(2*Nfft))*(randn(size(tx)) +
1j*randn(size(tx)));

            rx = h*tx + n;
            rx = rx(Lcp+1:end);
            Yf = fft(rx,Nfft)/h;

            rx_bits = qpsk_demapper(Yf);

            % Deinterleaver
            mat = reshape(rx_bits,16,32).';
            rx_bits = mat(:).';
```

```
            % Decoder
            rx_bits = rx_bits(1:5*length(info_bits));
            rx_bits = reshape(rx_bits,5,[]);
            dec_bits = sum(rx_bits) >= 3;

            err_cod_flat  = err_cod_flat + sum(dec_bits ~= info_bits);
            bits_cod_flat = bits_cod_flat + length(info_bits);

            %% ================================================
            %             UNCODED QPSK - FREQUENCY-SELECTIVE
            %% ================================================
            bits_uncoded = randi([0 1],1,512);
            Xf = qpsk_mapper(bits_uncoded,1);

            xt = ifft(Xf,Nfft);
            tx = [xt(end-Lcp+1:end) xt];

            % Independent Rayleigh per subcarrier
            h_freq = (randn(1,Nfft) + 1j*randn(1,Nfft))/sqrt(2);
            % Apply fading in frequency domain
             % Remove CP before FFT as CP is not part of the OFDM symbol
            tx_no_cp = tx(Lcp+1:end);
            faded = ifft(fft(tx_no_cp,Nfft).*h_freq,Nfft);
            tx_freq = [faded(end-Lcp+1:end) faded];

            n = sqrt(N0/(2*Nfft))*(randn(size(tx_freq)) +
1j*randn(size(tx_freq)));

            rx = tx_freq + n;
            rx_no_cp = rx(Lcp+1:end);
            Yf = fft(rx_no_cp,Nfft)./h_freq;

            demapped_bits = qpsk_demapper(Yf);
            err_unc_freq  = err_unc_freq + sum(demapped_bits ~=
bits_uncoded);
            bits_unc_freq = bits_unc_freq + length(bits_uncoded);

            %% ================================================
            %       CODED QPSK (RATE 1/5 - TX BIT) - FREQUENCY-SELECTIVE
            %% ================================================
            info_bits  = randi([0 1],1,floor(512/Rep));
            coded_bits = repelem(info_bits,Rep);
            coded_bits = [coded_bits 0 0];

            mat = reshape(coded_bits,32,16).';
            inter_bits = mat(:).';

            Xf = qpsk_mapper(inter_bits,1);

            xt = ifft(Xf,Nfft);
            tx = [xt(end-Lcp+1:end) xt];

            h_freq = (randn(1,Nfft) + 1j*randn(1,Nfft))/sqrt(2);
```

```
            tx_no_cp = tx(Lcp+1:end);
            faded = ifft(fft(tx_no_cp,Nfft).*h_freq,Nfft);
            tx_freq = [faded(end-Lcp+1:end) faded];

            n = sqrt(N0/(2*Nfft))*(randn(size(tx_freq)) +
1j*randn(size(tx_freq)));

            rx = tx_freq + n;
            rx_no_cp = rx(Lcp+1:end);
            Yf = fft(rx_no_cp,Nfft)./h_freq;

            rx_bits = qpsk_demapper(Yf);

            mat = reshape(rx_bits,16,32).';
            rx_bits = mat(:).';

            rx_bits = rx_bits(1:5*length(info_bits));
            rx_bits = reshape(rx_bits,5,[]);
            dec_bits = sum(rx_bits) >= 3;

            err_cod_freq  = err_cod_freq + sum(dec_bits ~= info_bits);
            bits_cod_freq = bits_cod_freq + length(info_bits);

        end

        BER_uncoded_flat(s) = err_unc_flat / bits_unc_flat;
        BER_coded_flat(s)   = err_cod_flat / bits_cod_flat;
        BER_uncoded_freq(s) = err_unc_freq / bits_unc_freq;
        BER_coded_freq(s)   = err_cod_freq / bits_cod_freq;
    end

    %% ================ PLOT ================
    figure; grid on; hold on;
    %ylim([1e-5 1e0]);
    semilogy(EbN0_dB,BER_uncoded_flat,'r-o','LineWidth',2);
    semilogy(EbN0_dB,BER_coded_flat,'b-s','LineWidth',2);
    semilogy(EbN0_dB,BER_uncoded_freq,'k--o','LineWidth',2);
    semilogy(EbN0_dB,BER_coded_freq,'m--s','LineWidth',2);
    xlabel('E_b / N_0 (dB)');
    ylabel('BER');
    ylim([1e-5 1]);
    legend('Uncoded Flat','Coded Flat (Rep 1/5, TX)','Uncoded Freq
Sel','Coded Freq Sel');
    title('OFDM QPSK - Flat & Frequency-Selective Rayleigh Channel');


    set(gca,'YScale','log');
    ylim([1e-4 1]);
    yticks([1e-4 1e-3 1e-2 1e-1 1]);



    %% ================ FUNCTIONS ================
    function symbols = qpsk_mapper(bits,A)
```

```
        b = reshape(bits,2,[]).';
        symbols = A*((2*b(:,1)-1) + 1j*(2*b(:,2)-1)).';
    end

    function bits = qpsk_demapper(symbols)
        bits = zeros(1,2*length(symbols));
        bits(1:2:end) = real(symbols) >= 0;
        bits(2:2:end) = imag(symbols) >= 0;
    end

end

%% part 6: (16QAM Scheme OFDM)
function part6()
    clc; clear; close all;

    %% ================ PARAMETERS ================
    Nfft = 256;           % IFFT size
    Lcp  = 16;            % Cyclic prefix length
    Rep  = 5;             % Repetition factor
    EbN0_dB = -5:2:15;    % SNR range in dB
    EbN0 = 10.^(EbN0_dB/10);

    Nsym = 2000;           % OFDM symbols per SNR point
    bits_per_symbol = 4;  % 16-QAM

    %% 16-QAM Constellation
    t_QAM = [-3-3i, -3-1i, -3+3i, -3+1i, ...
             -1-3i, -1-1i, -1+3i, -1+1i, ...
              3-3i,  3-1i,  3+3i,  3+1i, ...
              1-3i,  1-1i,  1+3i,  1+1i];

    %% ================ BER STORAGE ================
    BER_uncoded_flat = zeros(size(EbN0_dB));
    BER_coded_flat   = zeros(size(EbN0_dB));
    BER_uncoded_freq = zeros(size(EbN0_dB));
    BER_coded_freq   = zeros(size(EbN0_dB));

    %% ================ SNR LOOP ================
    for s = 1:length(EbN0_dB)

        err_unc_flat = 0; bits_unc_flat = 0;
        err_cod_flat = 0; bits_cod_flat = 0;
        err_unc_freq = 0; bits_unc_freq = 0;
        err_cod_freq = 0; bits_cod_freq = 0;

        N0 = 1 / EbN0(s);  % Eb = 1

        for sym = 1:Nsym
            %% ================================================
            %             UNCODED 16-QAM - FLAT
            %% ================================================
            bits_uncoded = randi([0 1],1, Nfft*bits_per_symbol);
            Xf = qam16_mapper(bits_uncoded, t_QAM, Nfft);
```

```
            xt = ifft(Xf, Nfft);
            xt = xt(:).';                       % Ensure row vector
            tx = [xt(end-Lcp+1:end), xt];   % add CP

            h = (randn + 1j*randn)/sqrt(2);
            n = sqrt(N0/(2*Nfft))*(randn(size(tx)) +
1j*randn(size(tx)));

            rx = h*tx + n;
            rx = rx(Lcp+1:end);
            Yf = fft(rx,Nfft)/h;

            demapped_bits = qam16_demapper(Yf, t_QAM);
            err_unc_flat  = err_unc_flat + sum(demapped_bits ~=
bits_uncoded);
            bits_unc_flat = bits_unc_flat + length(bits_uncoded);

            %% ================================================
            %          CODED 16-QAM (RATE 1/5 - TX BIT) - FLAT
            %% ================================================
            info_bits  = randi([0 1],1,
floor(Nfft*bits_per_symbol/Rep));
            coded_bits = repelem(info_bits,Rep);
            coded_bits = [coded_bits zeros(1, Nfft*bits_per_symbol -
length(coded_bits))];

            % Interleaver (32×32)
            mat = reshape(coded_bits,32,32).';
            inter_bits = mat(:).';

            Xf = qam16_mapper(inter_bits, t_QAM, Nfft);
            xt = ifft(Xf,Nfft);
            xt = xt(:).';                       % Ensure row vector
            tx = [xt(end-Lcp+1:end), xt];

            h = (randn + 1j*randn)/sqrt(2);
            n = sqrt(N0/(2*Nfft))*(randn(size(tx)) +
1j*randn(size(tx)));

            rx = h*tx + n;
            rx = rx(Lcp+1:end);
            Yf = fft(rx,Nfft)/h;

            rx_bits = qam16_demapper(Yf, t_QAM);

            % Deinterleaver
            mat = reshape(rx_bits,32,32).';
            rx_bits = mat(:).';

            % Decoder
            rx_bits = rx_bits(1:Rep*length(info_bits));
            rx_bits = reshape(rx_bits, Rep, []);
            dec_bits = sum(rx_bits) >= ceil(Rep/2);
```

```
            err_cod_flat  = err_cod_flat + sum(dec_bits ~= info_bits);
            bits_cod_flat = bits_cod_flat + length(info_bits);

            %% ================================================
            %         UNCODED 16-QAM - FREQUENCY-SELECTIVE
            %% ================================================
            bits_uncoded = randi([0 1],1, Nfft*bits_per_symbol);
            Xf = qam16_mapper(bits_uncoded, t_QAM, Nfft);

            xt = ifft(Xf,Nfft);
            xt = xt(:).';
            tx = [xt(end-Lcp+1:end), xt];

            h_freq = (randn(1,Nfft) + 1j*randn(1,Nfft))/sqrt(2);
            tx_no_cp = tx(Lcp+1:end);
            faded = ifft(fft(tx_no_cp,Nfft).*h_freq,Nfft);
            faded = faded(:).';
            tx_freq = [faded(end-Lcp+1:end), faded];

            n = sqrt(N0/(2*Nfft))*(randn(size(tx_freq)) +
1j*randn(size(tx_freq)));

            rx = tx_freq + n;
            rx_no_cp = rx(Lcp+1:end);
            Yf = fft(rx_no_cp,Nfft)./h_freq;

            demapped_bits = qam16_demapper(Yf, t_QAM);
            err_unc_freq  = err_unc_freq + sum(demapped_bits ~=
bits_uncoded);
            bits_unc_freq = bits_unc_freq + length(bits_uncoded);

            %% ================================================
            %       CODED 16-QAM - FREQUENCY-SELECTIVE
            %% ================================================
            info_bits  = randi([0 1],1,floor(Nfft*bits_per_symbol/Rep));
            coded_bits = repelem(info_bits,Rep);
            coded_bits = [coded_bits zeros(1, Nfft*bits_per_symbol -
length(coded_bits))];

            mat = reshape(coded_bits,32,32).';
            inter_bits = mat(:).';

            Xf = qam16_mapper(inter_bits, t_QAM, Nfft);
            xt = ifft(Xf,Nfft);
            xt = xt(:).';
            tx = [xt(end-Lcp+1:end), xt];

            h_freq = (randn(1,Nfft) + 1j*randn(1,Nfft))/sqrt(2);
            tx_no_cp = tx(Lcp+1:end);
            faded = ifft(fft(tx_no_cp,Nfft).*h_freq,Nfft);
            faded = faded(:).';
            tx_freq = [faded(end-Lcp+1:end), faded];
```

```
            n = sqrt(N0/(2*Nfft))*(randn(size(tx_freq)) +
1j*randn(size(tx_freq)));

            rx = tx_freq + n;
            rx_no_cp = rx(Lcp+1:end);
            Yf = fft(rx_no_cp,Nfft)./h_freq;

            rx_bits = qam16_demapper(Yf, t_QAM);
            mat = reshape(rx_bits,32,32).';
            rx_bits = mat(:).';

            rx_bits = rx_bits(1:Rep*length(info_bits));
            rx_bits = reshape(rx_bits, Rep, []);
            dec_bits = sum(rx_bits) >= ceil(Rep/2);

            err_cod_freq  = err_cod_freq + sum(dec_bits ~= info_bits);
            bits_cod_freq = bits_cod_freq + length(info_bits);
        end

        BER_uncoded_flat(s) = err_unc_flat / bits_unc_flat;
        BER_coded_flat(s)   = err_cod_flat / bits_cod_flat;
        BER_uncoded_freq(s) = err_unc_freq / bits_unc_freq;
        BER_coded_freq(s)   = err_cod_freq / bits_cod_freq;
    end

    %% ================ PLOT ================
    figure; grid on; hold on;
    semilogy(EbN0_dB,BER_uncoded_flat,'r-o','LineWidth',2);
    semilogy(EbN0_dB,BER_coded_flat,'b-s','LineWidth',2);
    semilogy(EbN0_dB,BER_uncoded_freq,'k--o','LineWidth',2);
    semilogy(EbN0_dB,BER_coded_freq,'m--s','LineWidth',2);
    xlabel('E_b / N_0 (dB)');
    ylabel('BER');
    legend('Uncoded Flat','Coded Flat (Rep 1/5)','Uncoded Freq
Sel','Coded Freq Sel');
    title('OFDM 16-QAM - Flat & Frequency-Selective Rayleigh Channel');

    set(gca,'YScale','log');
    ylim([1e-4 1]);
    yticks([1e-4 1e-3 1e-2 1e-1 1]);

    %% ================ FUNCTIONS ================
    function symbols = qam16_mapper(bits, t_QAM, Nfft)
        bits_per_symbol = 4;
        bits = bits(1:bits_per_symbol*Nfft);
        b = reshape(bits, bits_per_symbol, []).';
        idx = bi2de(b,'left-msb') + 1;
        symbols = t_QAM(idx).';
    end

    function bits = qam16_demapper(symbols, t_QAM)
        bits = zeros(1, 4*length(symbols));
        for k = 1:length(symbols)
            [~, idx] = min(abs(symbols(k) - t_QAM.'));
```

```
            bits(4*(k-1)+1:4*k) = de2bi(idx-1,4,'left-msb');
        end
    end

end
```